

Reglas de Agrupación de Datos Numéricos - Solución

Pregunta 1

Da el algoritmo para reproducir el proceso de generación de clases y sus marcas donde el número de clases ha sido obtenido con la regla de la Scott en **Python**.

Solución

```
import math
import numpy as np
import statistics

# x = vector de datos de la muestra
# n = Número total de datos de la muestra

# Número de clases por regla de Scott
As = 3.5 * statistics.stdev(x) * math.pow(n, -1. / 3.)
k = math.ceil((max(x) - min(x)) / As) # Número de clases por regla de Scott

# precision = Precisión de la medida de los datos
# Calculamos la amplitud
A = n / k # Amplitud
# Redondeamos por exceso A según las cifras decimales de la precisión
A = round(A, dec_precision)

# Calculamos los extremos de los intervalos
L = np.zeros(k + 1)
L[0] = min(x) - 1 / 2 * precision
for i in range(1, k + 1):
    L[i] = L[0] + i * A

# Calculamos las marcas de clase
X = np.zeros(k)
for i in range(k):
    X[i] = (L[i] + L[i + 1]) / 2
```

Pregunta 2

Da el algoritmo para reproducir el proceso de generación de clases y sus marcas donde el número de clases ha sido obtenido con la regla de la raíz en **Python**.

Solución

```
import math
import numpy as np

# x = vector de datos de la muestra
# n = Número total de datos de la muestra

# Número de clases por regla de la raíz
k = math.ceil(math.sqrt(n))

# precision = Precisión de la medida de los datos
# Calculamos la amplitud
A = n / k # Amplitud
# Redondeamos por exceso A según las cifras decimales de la precisión
A = round(A, dec_precision)

# Calculamos los extremos de los intervalos
L = np.zeros(k + 1)
L[0] = min(x) - 1 / 2 * precision
for i in range(1, k + 1):
    L[i] = L[0] + i * A

# Calculamos las marcas de clase
X = np.zeros(k)
for i in range(k):
    X[i] = (L[i] + L[i + 1]) / 2
```

Pregunta 3

Da el algoritmo para reproducir el proceso de generación de clases y sus marcas donde el número de clases ha sido obtenido con la regla de la Sturges en Python.

Solución

```
import math
import numpy as np

# x = vector de datos de la muestra
# n = Número total de datos de la muestra

# Número de clases por regla de la raíz
k = math.ceil(math.sqrt(n))

# precision = Precisión de la medida de los datos
# Calculamos la amplitud
A = n / k # Amplitud
# Redondeamos por exceso A según las cifras decimales de la precisión
A = round(A, dec_precision)
```

```

# Calculamos los extremos de los intervalos
L = np.zeros(k + 1)
L[0] = min(x) - 1 / 2 * precision
for i in range(1, k + 1):
    L[i] = L[0] + i * A

# Calculamos las marcas de clase
X = np.zeros(k)
for i in range(k):
    X[i] = (L[i] + L[i + 1]) / 2

```

Pregunta 4

Da el algoritmo para reproducir el proceso de generación de clases y sus marcas donde el número de clases ha sido obtenido con la regla de la Freedman-Diaconis en Python.

Solución

```

import math
import numpy as np
from scipy.stats.mstats import mquantiles as quantiles

# x = vector de datos de la muestra
# n = Número total de datos de la muestra

# Número de clases por regla de Freedman-Diaconis
q = quantiles(x)
Afd = 2 * (q[2] - q[0]) * math.pow(n, -1. / 3.)
k = math.ceil((max(x) - min(x)) / Afd)

# precision = Precisión de la medida de los datos
# Calculamos la amplitud
A = n / k # Amplitud
# Redondeamos por exceso A según las cifras decimales de la precisión
A = round(A, dec_precision)

# Calculamos los extremos de los intervalos
L = np.zeros(k + 1)
L[0] = min(x) - 1 / 2 * precision
for i in range(1, k + 1):
    L[i] = L[0] + i * A

# Calculamos las marcas de clase
X = np.zeros(k)
for i in range(k):
    X[i] = (L[i] + L[i + 1]) / 2

```